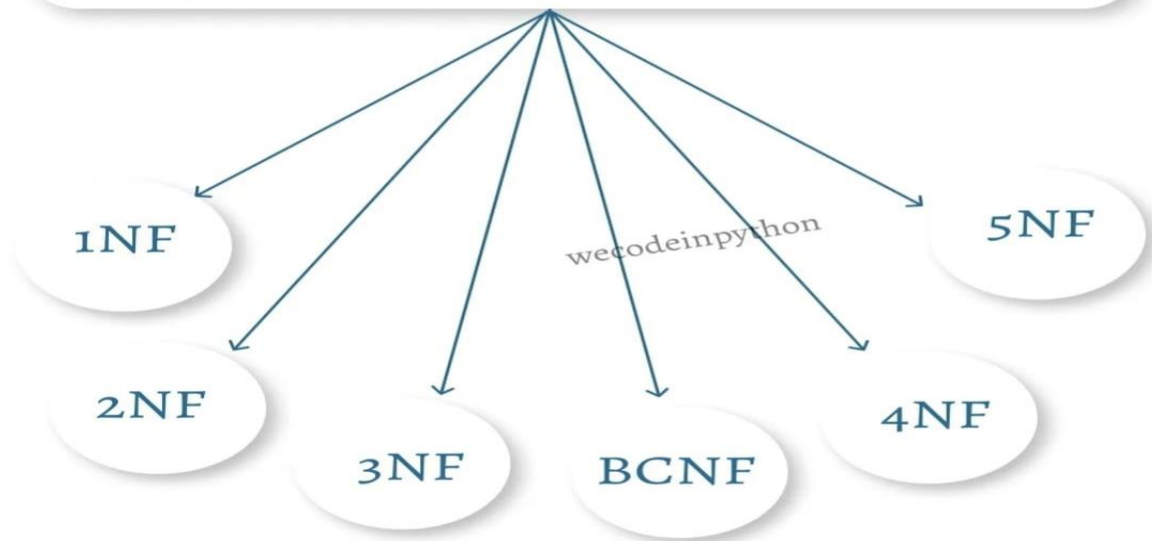


Normalization

- Normalization is the process of organizing the data in the database.
- Normalization is used to minimize the redundancy from a relation or set of relations. It is also used to eliminate the undesirable characteristics like Insertion, Update and Deletion Anomalies.
- Normalization divides the larger table into the smaller table and links them using relationship.
- The normal form is used to reduce redundancy from the database table.

wecodeinpython

Types of Normalization



First Normal Form(1NF)

- A relation will be 1NF if it contains an atomic value.
- It states that an attribute of a table cannot hold multiple values. It must hold only single-valued attribute.
- First normal form disallows the multi-valued attribute, composite attribute, and their combinations

Employee table

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
1	John	012345679, 121232493	Delhi
2	Ben	213123433	UP
3	Dan	342141244, 132412344	MP

wecodeinpython

EMP_ID	EMP_NAME	EMP_PHONE	EMP_STATE
1	John	123456789	Delhi
1	John	121232493	Delhi
2	Ben	213123433	UP
3	Dan	342141244	MP
3	Dan	132412344	MP

Second Normal Form(2NF)

- In the 2NF, relational must be in 1NF.
- In the second normal form, all non-key attributes are fully functional dependent on candidate key.

Student_id	Subject_id	Subject	Teacher	Marks
1	1	Java	Mr. Java	80
1	2	Python	Mr. Python	90
2	1	Java	Mr. Java	70
2	2	Python	Mr. Python	80

here marks are dependent upon student_id and subject_id so primary key will be combination of both but for subject name and teacher they are only dependent upon subject id (partial dependency) so it is not in 2NF. we can make it in 2nf by creating another separate table for subject teacher.

Student_id	Subject_id	Marks
1	1	80
1	2	90
2	1	70
2	2	80

Subject_id	Subject	Teacher
1	Java	Mr. Java
2	Python	Mr. Python

wecodeinpython

Third Normal Form(3NF)

- A relation will be in 3NF if it is in 2NF and not contain any transitive partial dependency.
- 3NF is used to reduce the data duplication. It is also used to achieve the data integrity.
- If there is no transitive dependency for non-prime attributes, then the relation must be in third normal form

Match_No	Teams	Stadium	Capacity
1	IND vs ENG	Wankhede	80K
2	PAK vs IND	Eden Gardens	90K
3	ENG vs AUS	JLN	70K

in this example match no is primary and Teams and Stadium is dependent upon match no but Capacity is dependent upon Stadium (Transitive dependency) so this is not in 3NF. we can make it in 3NF by create another table for capacity

Match_No	Teams	Stadium
1	IND vs ENG	Wankhede
2	PAK vs IND	Eden Gardens
3	ENG vs AUS	JLN

Stadium	Capacity
Wankhede	80K
Eden Gardens	90K
JLN	70K



wecodeinpython

Normalization (Database Normalization) is the process of organizing data in a database to reduce redundancy (duplicate data) and improve data integrity.

Types of Normalization

1. First Normal Form (1NF)

Rule: Each column should contain atomic (single) values, and there should be no repeating groups.

Before 1NF

StudentID	Name	Subjects
101	Amit	Math, Science
102	Neha	English, History

Here, `Subjects` contains multiple values.

After 1NF

StudentID	Name	Subject
101	Amit	Math
101	Amit	Science
102	Neha	English
102	Neha	History

2. Second Normal Form (2NF)

Rule: Must be in 1NF and all non-key attributes must depend on the entire primary key.

Before 2NF

StudentID	CourseID	StudentName	CourseName
101	C1	Amit	DBMS
102	C2	Neha	Java

Primary Key = (StudentID, CourseID)

`StudentName` depends only on `StudentID`, and `CourseName` depends only on `CourseID`.

After 2NF

Students Table

StudentID StudentName

101	Amit
102	Neha

Courses Table**CourseID CourseName**

C1	DBMS
C2	Java

Enrollments Table**StudentID CourseID**

101	C1
102	C2

3. Third Normal Form (3NF)

Rule: Must be in 2NF and there should be no transitive dependency (non-key attribute depending on another non-key attribute).

Before 3NF**EmpID EmpName DeptID DeptName**

E1	Rahul	D1	HR
E2	Priya	D2	Finance

DeptName depends on DeptID, not directly on EmpID.

After 3NF**Employee Table****EmpID EmpName DeptID**

E1	Rahul	D1
E2	Priya	D2

Department Table**DeptID DeptName**

D1	HR
D2	Finance

4. Boyce-Codd Normal Form (BCNF)

Rule: For every functional dependency, the determinant must be a candidate key.

Example

Teacher Subject Room

A	DBMS	R1
A	Java	R2
B	DBMS	R1

Dependencies:

- Teacher $\rightarrow$ Room
- Subject $\rightarrow$ Room

Since `Subject` is not a candidate key, BCNF is violated.

After BCNF

Subject_Room

Subject Room

DBMS	R1
Java	R2

Teacher_Subject

Teacher Subject

A	DBMS
A	Java
B	DBMS

Summary

Normal Form	Purpose
1NF	Remove repeating groups and multi-valued attributes
2NF	Remove partial dependency
3NF	Remove transitive dependency
BCNF	Remove anomalies caused by candidate keys
4NF	Remove multi-valued dependencies
5NF	Remove join dependencies

Memory Trick:

- **1NF** → Atomic values
- **2NF** → No Partial Dependency
- **3NF** → No Transitive Dependency
- **BCNF** → Every determinant is a candidate key
- **4NF** → No Multi-valued Dependency
- **5NF** → No Join Dependency